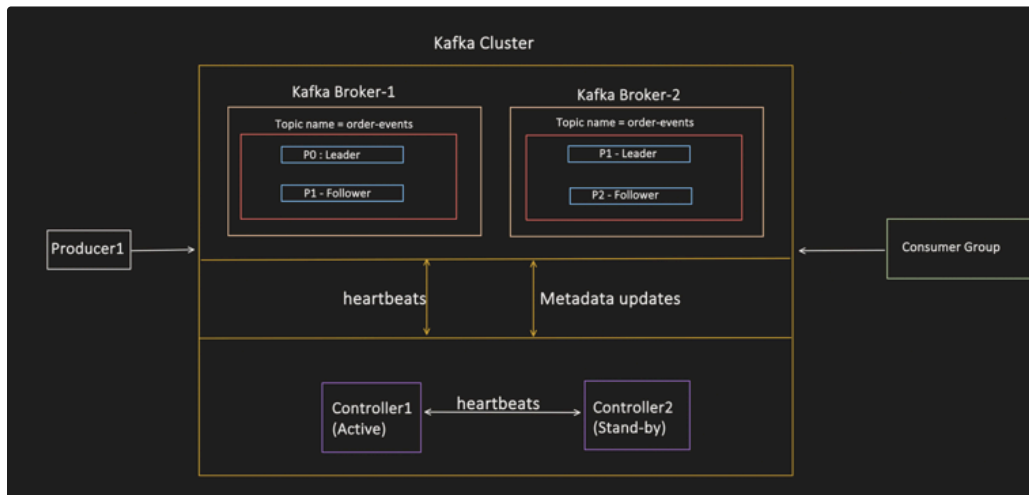
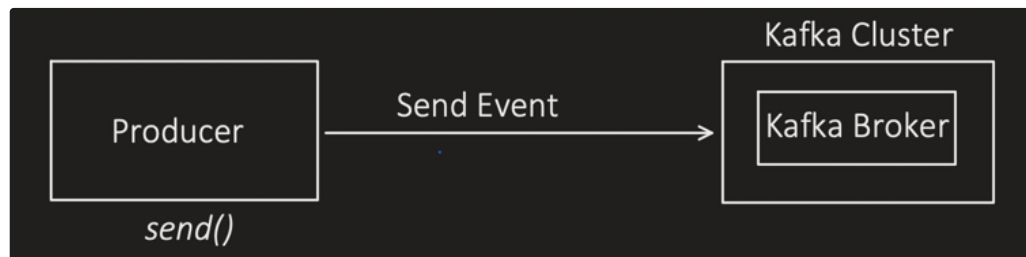


Producer Internals and Setup

So we have already setup the Kafka Cluster:

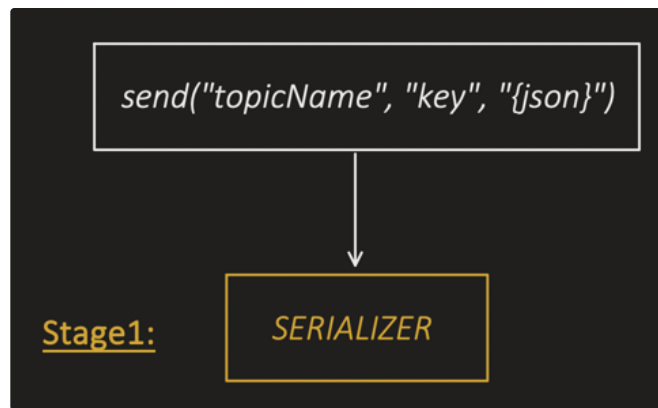


Before we proceed with Producer setup, let's understand the producer workflow:

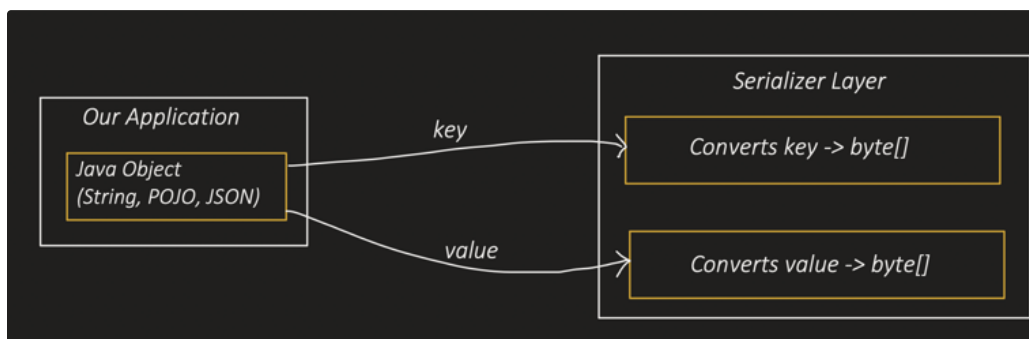


- When Producer try to send a message, it does not immediately send it to a broker.
- Instead message goes through multiple internal stages inside the producer, before any network call happens.

Let understand what steps are actually involved behind the scene:



- Our message (event) is generally a Java Object but Kafka understands only bytes. That's why Serializer is required.
- Kafka does not know, what is the data type of our message (Integer, String, Custom POJO, JSON etc.). That's why it want us to choose proper SERIALIZER.
- If Serialization stage fails, message will not move to next stage.



Every serializer implements this interface:

```

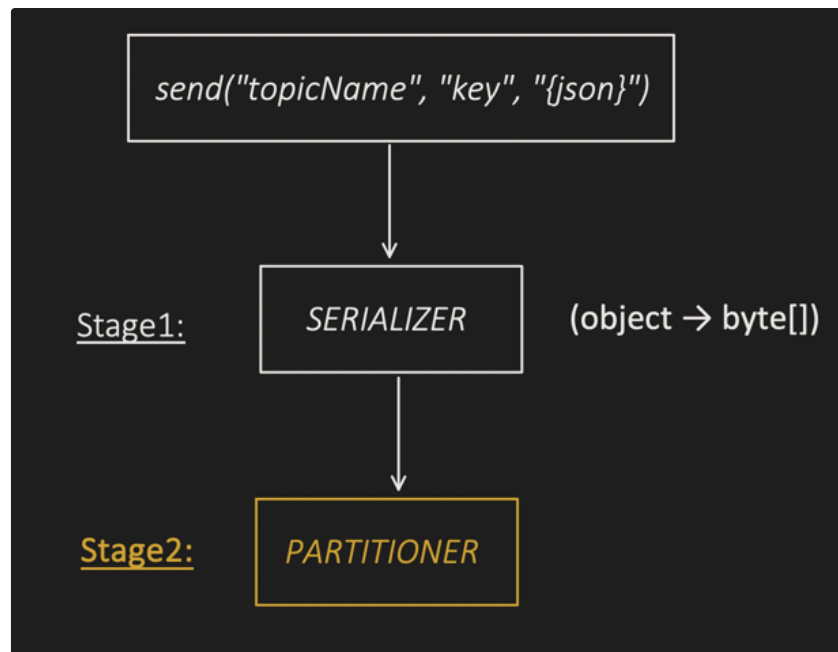
1 public interface Serializer<T> {
2     byte[] serialize(String topic, T data);
3 }
4 
```

There are many built-in Serializer, some of them are:

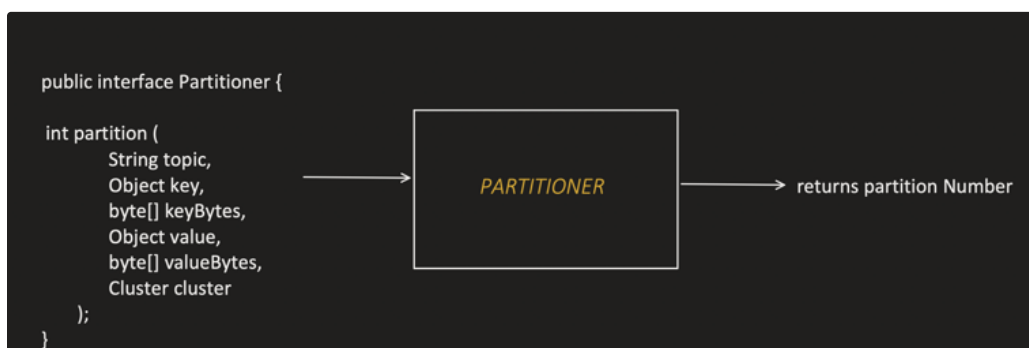
Serializer	Java Type
StringSerializer	String

IntegerSerializer	Integer
LongSerializer	Long
DoubleSerializer	Double
FloatSerializer	Float
ShortSerializer	Short
JsonSerializer	Any POJO (most useful, converts java object to bytes)
Custom Serializer	When we want full control: • encrypt the data before byte conversion. • Removing sensitive fields before sending. etc.

We will see this with an example, when we will setup the producer.



At this stage, producer decides "Which partition should this message go to?"



Case 1 : Partition explicitly provided

We can explicitly provide the partition no, so in this scenario, Partitioner does nothing.

KafkaTemplate.java

```

1  @Override
2  public CompletableFuture<SendResult<K, V>> send(String topic, Integer
   partition, K key, @Nullable V data) {
3      ProducerRecord<K, V> producerRecord = new
   ProducerRecord<>(topic, partition, key, data);
4      return observeSend(producerRecord);
5  }
6

```

Case 2 : Key is provided

Kafka ensures same key always goes to same partition.

KafkaTemplate.java

```
1 @Override
2 public CompletableFuture<SendResult<K, V>> send(String topic, K key,
3     @Nullable V data) {
4     ProducerRecord<K, V> producerRecord = new ProducerRecord<>
5     (topic, key, data);
6     return observeSend(producerRecord);
7 }
```

partition = hash(keyBytes) % numberOfPartitions

Case 3 : No Key is provided

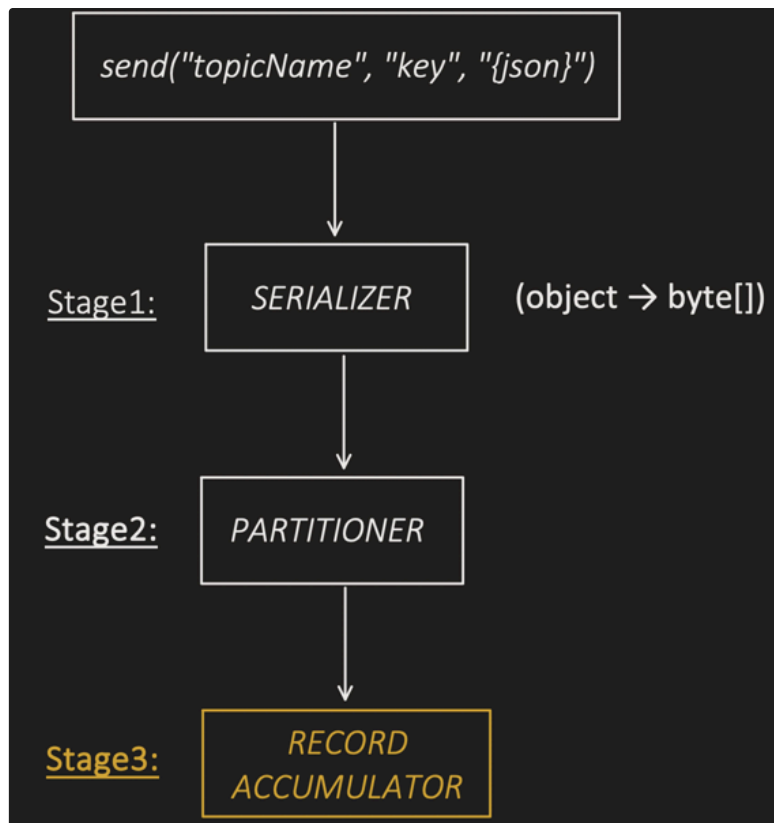
KafkaTemplate.java

```
1 @Override
2 public CompletableFuture<SendResult<K, V>> send(String topic, @Nullable
3     V data) {
4     ProducerRecord<K, V> producerRecord = new ProducerRecord<>
5     (topic, data);
6     return observeSend(producerRecord);
7 }
```

Pre Kafka 2.4: Round Robin Strategy

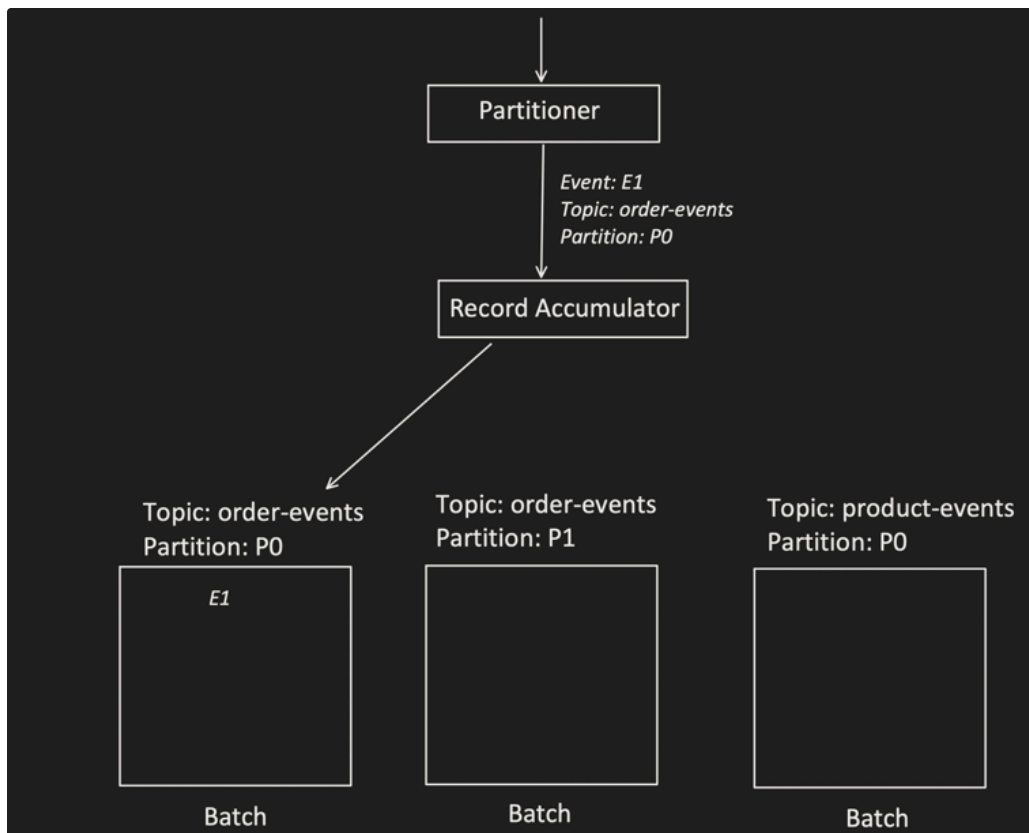
From Kafka 2.4 onwards: Sticky Partitioning Strategy

Whats the difference? Will explain after next stage



In record Accumulator:-

- In-memory buffer inside Kafka Producer, that collects events per topic-partition and group them into batches.
- And because of this only, Kaka Producer achieve very high throughput (ability to transmit large volume of data within a given timeframe)



So, now we know that, Producer always maintains topic-partition wise batch, and it sends batch to the broker (not individual events).

Now let's come back to the point:

Pre Kafka 2.4: Round Robin Partitioning Strategy

From Kafka 2.4 onwards: Sticky Partitioning Strategy

Example:

Topic: order-events, Key = null, Value = E1

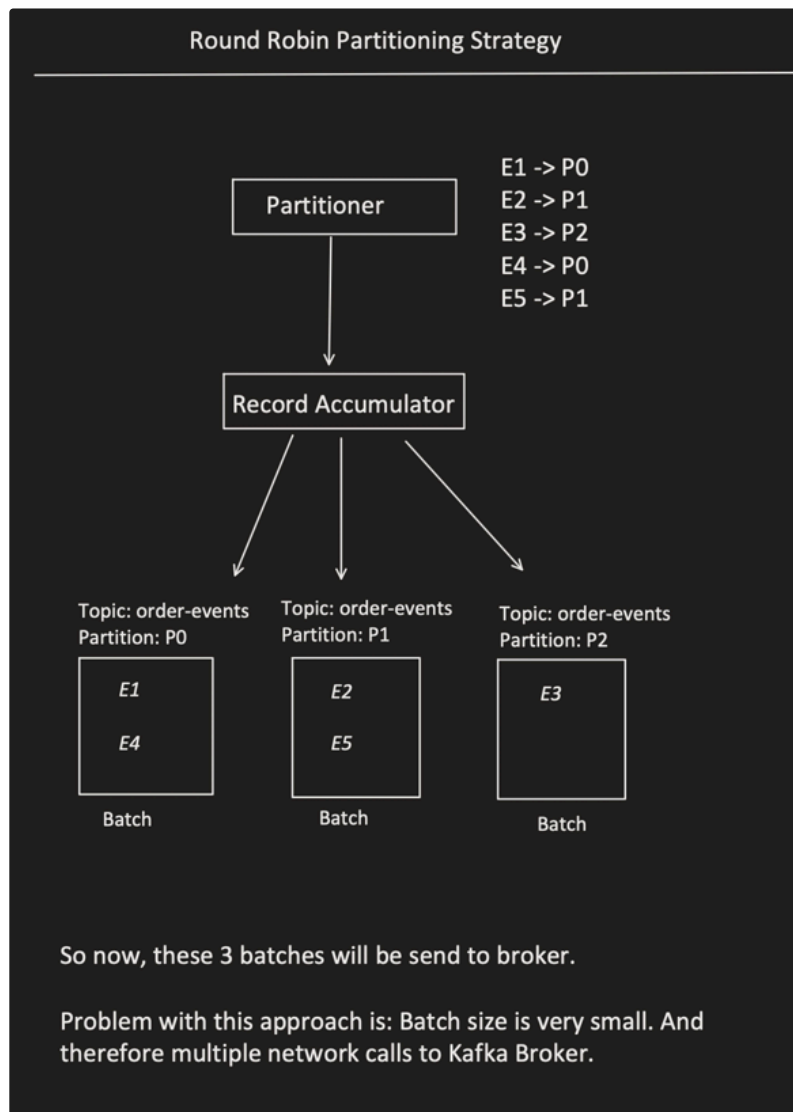
Topic: order-events, Key = null, Value = E2

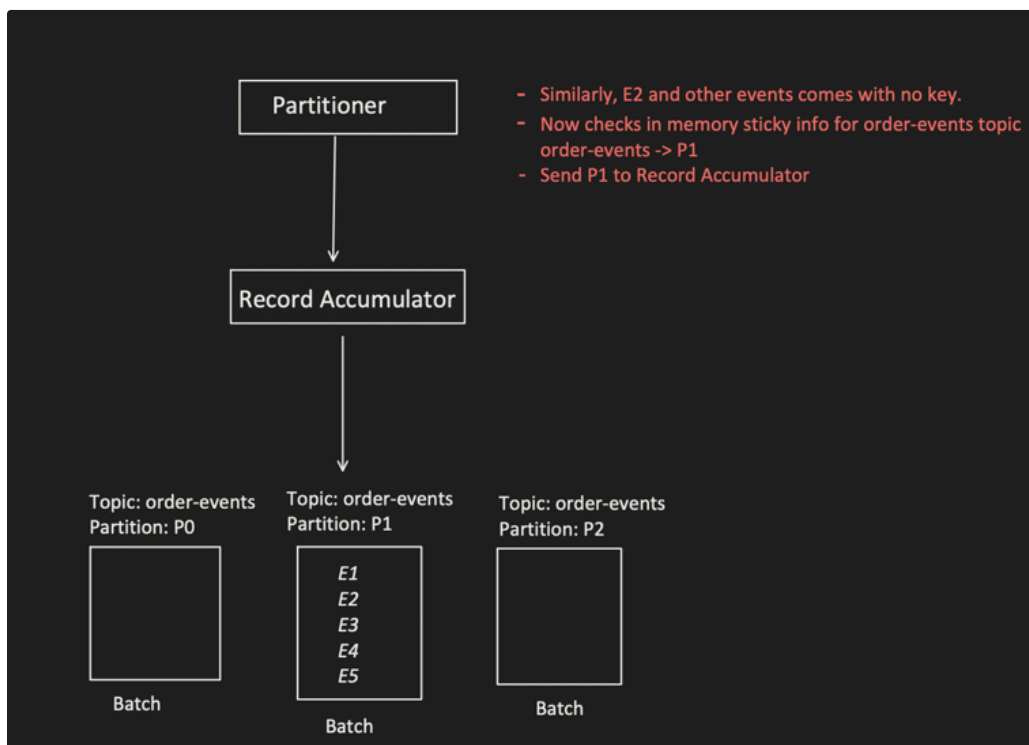
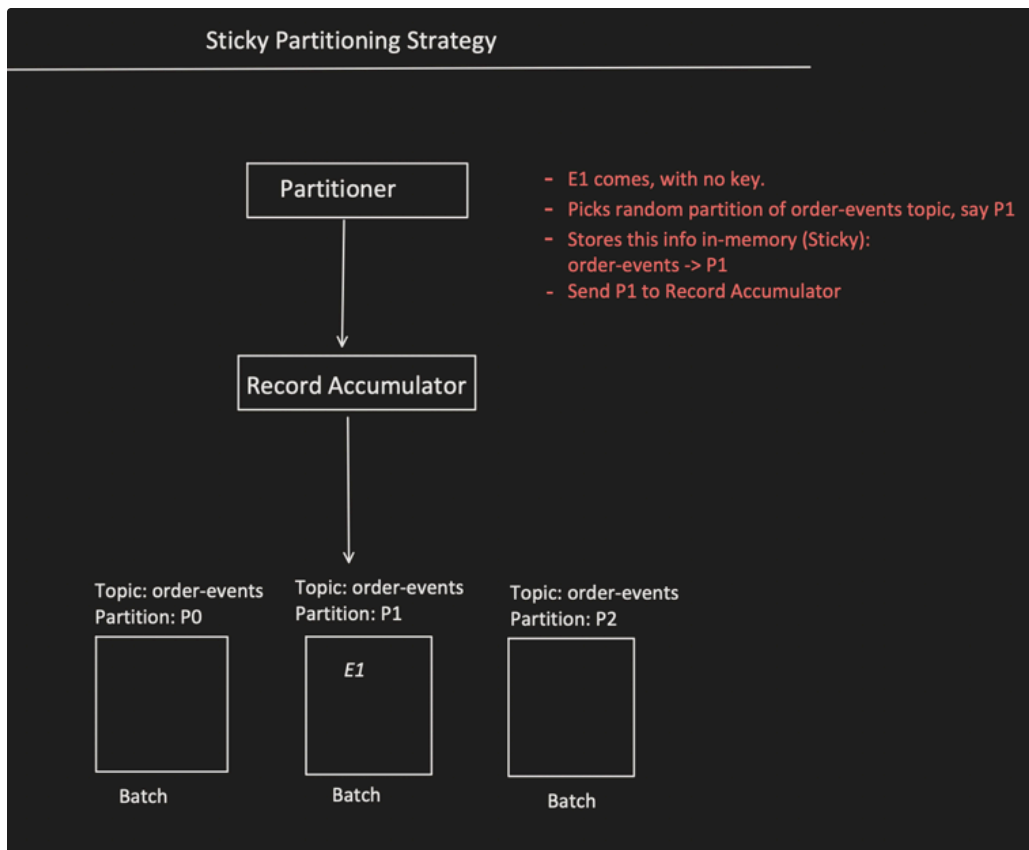
Topic: order-events, Key = null, Value = E3

Topic: order-events, Key = null, Value = E4

Topic: order-events, Key = null, Value = E5

Number of Partitions for order-events topic : [P0, P1, P2]





Advantage:

Events stick to the same partition **until the current batch is closed or a new batch needs to be created..**

So less number of batch and therefore less number of network calls.

Now question we might get is, "**How to control the batch?**"

There are 2 ways to control the batch:-

1. **batch.size**=16384 #16KB (default)
2. **linger.ms**=10ms

Now in batch.size=16384 #16KB (default)

- Means maximum memory allocated per partition batch.
- For example :
if 1 Event size = 1KB then

Batch will hold ~16 messages.

and when batch size is reached to its limit, batch is closed and its reading for sending.

In linger.ms=10ms

- Maximum time producer will wait for more records to fill the batch.
- This make sure that, producer do not wait indefinitely for a batch to fill when traffic is low.

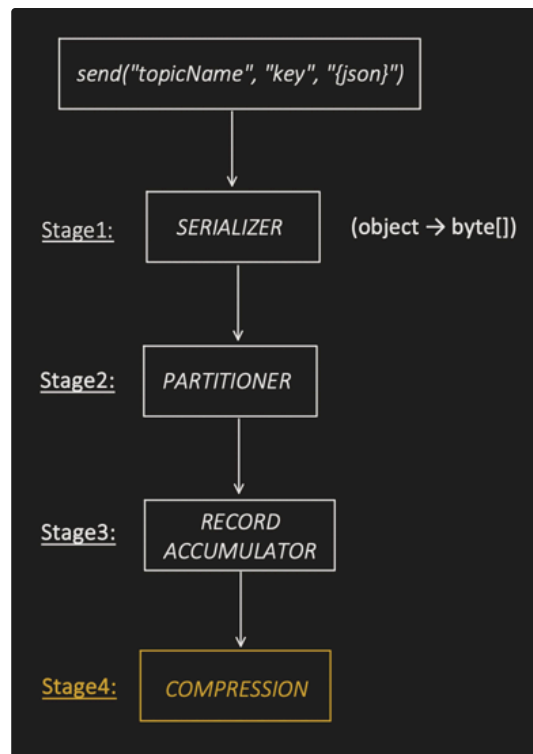
For example:

Batch size = 16KB

Event size = 1KB

means 1 batch can hold 16 messages. But traffic is less and only 5 events yet added in batch.

Producer will max wait 10ms for more events, else after 10ms it close the batch and it will be ready to send.



- Batches which are ready to send are compressed to reduce network usage and thus achieve higher throughput.

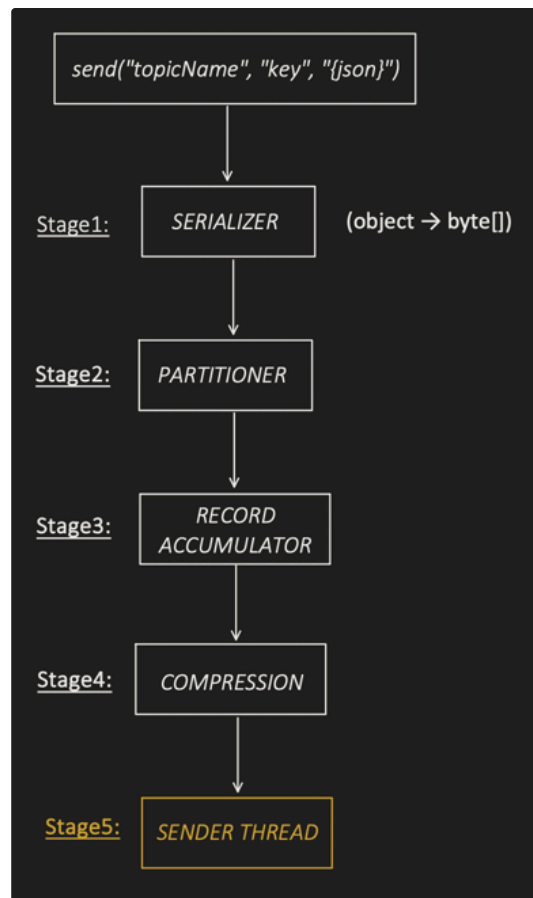
Ex:

Batch before compression = 64KB

After compression = 18KB

- Kafka stores these compressed batches thus save disk space too.

When we will setup the producer, will show how in properties we can add this compression.



- Application thread never send data to Kafka broker.
- Background sender thread does the actual communication.

Application thread work is till writing the record to Record Accumulator, after that sender thread comes into the picture:

Sender thread Responsibility:

1. check RecordAccumulator for ready batches
2. Group batches by broker leader. Ex:
 - a. Batch for TopicA-Partition0 → Leader is Broker1
 - b. Batch for TopicA-Partition5 → Leader is Broker1
 - c. Batch for TopicB-Partition1 → Leader is Broker1

So, broker leader wise it will group the batches and wrap it up in 1 `ProduceRequest` and send it in 1 network call.

1. send request using NetworkClient
2. process responses (acks, retries)
3. Handle retries if needed
4. Complete future and invoke callbacks.

Now question might comes to you, can we control these Sender Thread like how many they should exist etc.

Answer is **NO**, we don't have a way to control this. Kafka manage it internally.

But wait, there is 1 thing which we can control :

Say we have 3 Brokers, so Sender Thread will create 3 TCP connection (one for each) to send the message to the respective broker.

`max.in.flight.requests.per.connection = 5` (default)

This parameter controls the maximum number of unacknowledged requests the client will send on a single connection before blocking.

So we have 3 TCP connection, so max 15 requests can be in-flight across the cluster at once.

But very important:

`max.in.flight.requests.per.connection > 1`

and

Retries is also `> 0`

Then there is risk of message reordering.

Ex:

Request A → in-flight

Request B → in-flight

Request A → fails

Request B → success (written successfully in partition)

Request A → retries and Success (written after Request B)

This Risk is resolved through **IDEMPOTENCY** set to TRUE, which we will see separately.

Now, we know a bit about Producer, let's start our Producer Setup:

Step1: Dependency.

pom.xml

```
1 <dependency>
2   <groupId>org.springframework.kafka</groupId>
3   <artifactId>spring-kafka</artifactId>
4 </dependency>
5
```

This single dependency pulls:

spring-kafka = **Spring wrapper layer + Kafka client dependency.**

Spring wrapper layer:

- **KafkaTemplate** - which is required to send messages
- **KafkaAdmin** - which is required to manage topic

etc. These are Spring abstraction build on top of Apache kafka client.

And it also bring actual Kafka client library. The real implementation which KafkaTemplate, KafkaAdmin etc. uses.

Step2: Add Bootstrap Brokers

```
1 server.port=8081
2 spring.application.name=kafka-producer-service
3
4 #Both brokers: if one is down, producer connects to the other
5 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
6
7 #Metadata refresh interval, default is 5minutes only
```

```
8 spring.kafka.producer.properties.metadata.max.age.ms=300000
9
10
```

This bootstrap info is required because say initially producer do not know, which broker is leader for which partition or for topic creation it can not directly talk with controller, so it need 1 starting communication point.

- *If it's a topic creation request, that broker will pass the request to active controller.*
- *If its regarding cluster metadata fetch, if broker has the info it will send else it will fetch from controller and send it back to producer.*
- *Producer will store it in-memory and keep refresh it after 5 minutes (default) or if any issue comes during connection with the leader of the partition.*

Step3: Topic Creation

```
1 @Configuration
2 public class KafkaProducerConfig {
3
4     @Bean
5     public NewTopic orderEventsTopic() {
6
7         return TopicBuilder.name("order-events").build();
8     }
9 }
10
```

We have seen in Cluster setup part that, if during topic creation, partition and replication factor information is not provided, then from controller properties default values will be picked.

Similarly we can provide default value of Segment size, clean up policy in Broker properties.

Same config we can also provide during Topic creation too, TopicBuilder provide us a way to set the configurations, we have so many configuration, showing some important configuration below:

```
1
2 @Bean
3 public NewTopic orderEventsTopic() {
4
5     return TopicBuilder.name("order-events")
6
7         //if not provided here, will be picked from controller
properties
```

```

8      .partitions(3)
9      .replicas(2)
10
11      // if not provided here, will be picked from broker
12      properties
13      .config(TopicConfig.RETENTION_MS_CONFIG, "604800000")
14
15      // if not provided here, will be picked from broker
16      properties
17      .config(TopicConfig.CLEANUP_POLICY_CONFIG, "delete")
18
19      // if not provided here, will be picked from broker
20      properties
21      .config(TopicConfig.MIN_IN_SYNC_REPLICAS_CONFIG, "2")
22
23      // if not provided here, will be picked from broker
24      properties
25      .config(TopicConfig.SEGMENT_BYTES_CONFIG,
26      "1073741824")
27
28      .build();
29  }

```

During application startup, it will check if specific topic exist or not, if not then it will send the create topic request.

Step4: Send Event

Usecase1:

• *SERIALIZER:*

- Key : String
- Value : JSON

• *PARTITIONER:*

- Send with Key, so that related events goes to same partition.

• *RECORD ACCUMULATOR*

- Max Batch size: 32KB
- Linger = 20ms
- Overall Buffer for batches = 32MB
- When Overall Buffer is full, Send() message should wait for 60sec after that start throwing exception.

• COMPRESSION

- Use one out of many : snappy, gzip, lz4, zstd etc.

• SENDER THREAD

- In-flight request per connection = 5
- 3 times retry on transient failures

OrderController.java

```
1 @RestController
2 @RequestMapping("/api/orders")
3 public class OrderController {
4
5     @Autowired
6     OrderProducerService orderProducerService;
7
8     @PostMapping("/with-key")
9     public ResponseEntity<String> sendWithKey(@RequestBody Order
10 order) {
11         orderProducerService.sendWithKey(order);
12         return ResponseEntity.accepted().body("Order created and
13 event published!");
14     }
15 }
```

OrderProducerService.java

```
1 import org.springframework.beans.factory.annotation.Autowired;
2 import org.springframework.kafka.core.KafkaTemplate;
3 import org.springframework.kafka.support.SendResult;
4 import org.springframework.stereotype.Service;
5 import java.util.concurrent.CompletableFuture;
6
7 @Service
8 public class OrderProducerService {
9
10     @Autowired
11     private KafkaTemplate<String, Order> kafkaTemplate;
12
13     public void sendWithKey(Order order) {
14         String key = order.getOrderid();
15
16         CompletableFuture<SendResult<String, Order>> future =
17             kafkaTemplate.send("order-events", key, order);
18
19         future.whenComplete((result, ex) -> {
20             if (ex == null) {
21                 System.out.println("Order event sent
22 successfully");
23             }
24         });
25     }
26 }
```

```

21         System.out.println("Partition: " +
result.getRecordMetadata().partition());
22         System.out.println("Offset: " +
result.getRecordMetadata().offset());
23     } else {
24         System.out.println("Failed to send order
event: " + ex.getMessage());
25     }
26     });
27 }
28 }
29

```

Order.java (POJO class)

```

1 public class Order {
2     private String orderId;
3     private String customerId;
4     private String productId;
5     private Integer quantity;
6     private Double totalAmount;
7     private String status;
8
9     //getters and setters
10 }
11

```

application.properties

```

1 server.port=8081
2 spring.application.name=kafka-producer-service
3
4 # Both brokers listed for redundancy, if one is down, producer
connects to the other
5 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
6
7 # Metadata refresh interval: how often producer re-fetches cluster
info (default 5 min)
8 spring.kafka.producer.properties.metadata.max.age.ms=300000
9
10 # Asking broker to reply only when all ISR successfully get the events
11 spring.kafka.producer.acks=all
12
13 #-----SERIALIZER-----
14 # Key needs to be String
15 spring.kafka.producer.key-
serializer=org.apache.kafka.common.serialization.StringSerializer
16
17 # Value: JsonSerializer, auto-converts Java objects to JSON bytes
18 spring.kafka.producer.value-
serializer=org.springframework.kafka.support.serializer.JsonSerializer
19
20
21 #-----RECORD ACCUMULATOR-----
22 # Batching, wait 20ms to collect more messages before sending
23 spring.kafka.producer.properties.linger.ms=20
24
25 # Max batch size, send when batch reaches 32KB (even if linger.ms
hasn't expired)
26 spring.kafka.producer.properties.batch.size=32768

```

```

27
28 # Total buffer memory for all batches: 32MB (if full, send() will get
   blocked till space created or max block time)
29 spring.kafka.producer.properties.buffer.memory=33554432
30
31 # Max time send() blocks if buffer is full (default 60s), after this
   time, send() call will fail immediately
32 spring.kafka.producer.properties.max.block.ms=60000
33
34
35 #-----COMPRESSION-----
36 # compresses entire batch before sending
37 spring.kafka.producer.properties.compression.type=snappy
38
39
40 #-----SENDER THREAD-----
41 # Retry on transient failures (network timeout, leader change, etc.)
42 spring.kafka.producer.retries=3
43
44 # Max in-flight requests
45 spring.kafka.producer.properties.max.in.flight.requests.per.connection
   =5
46

```

Usecase2:

• **SERIALIZER:**

- Key : String
- Value : Custom Serializer

• **PARTITIONER:**

- Send without Key (Sticky Partitioning)

Rest all same like previous

```

1  @RestController
2  @RequestMapping("/api/orders")
3  public class OrderController {
4
5      @Autowired
6      OrderProducerService orderProducerService;
7
8      @PostMapping("/with-no-key")
9      public ResponseEntity<String> sendWithoutKey(@RequestBody Order
   order) {
10         orderProducerService.sendWithoutKey(order);
11         return ResponseEntity.accepted().body("Order created and
   event published!");
12     }
13 }
14

```

```

1  import org.springframework.beans.factory.annotation.Autowired;
2  import org.springframework.kafka.core.KafkaTemplate;
3  import org.springframework.kafka.support.SendResult;
4  import org.springframework.stereotype.Service;
5  import java.util.concurrent.CompletableFuture;
6
7  @Service
8  public class OrderProducerService {
9
10     @Autowired
11     private KafkaTemplate<String, Order> kafkaTemplate;
12
13     public void sendWithoutKey(Order order) {
14         String key = order.getOrderid();
15
16         CompletableFuture<SendResult<String, Order>> future =
17             kafkaTemplate.send("order-events", order);
18
19         future.whenComplete((result, ex) -> {
20             if (ex == null) {
21                 System.out.println("Order event sent
22                 successfully");
23                 System.out.println("Partition: " +
24                 result.getRecordMetadata().partition());
25                 System.out.println("Offset: " +
26                 result.getRecordMetadata().offset());
27             } else {
28                 System.out.println("Failed to send order
29                 event: " + ex.getMessage());
30             }
31         });
32     }
33 }

```

Custom Serializer:

```

1  import com.fasterxml.jackson.core.JsonProcessingException;
2  import com.fasterxml.jackson.databind.ObjectMapper;
3  import org.apache.kafka.common.serialization.Serializer;
4  import java.util.LinkedHashMap;
5  import java.util.Map;
6
7  public class OrderSummarySerializer implements Serializer<Order> {
8
9     private final ObjectMapper objectMapper;
10
11     public OrderSummarySerializer() {
12         this.objectMapper = new ObjectMapper();
13     }
14
15     @Override
16     public byte[] serialize(String topic, Order order) {
17         if (order == null) {
18             return null;
19         }
20
21         try {
22             // Build a map with only the fields we want to expose,
23             // rest I removed it
24             Map<String, Object> summary = new LinkedHashMap<>();
25             summary.put("orderId", order.getOrderid());
26             summary.put("productId", order.getProductId());
27
28         } catch (JsonProcessingException e) {
29             // Handle exception
30         }
31     }
32 }

```

```

26         return objectMapper.writeValueAsBytes(summary);
27
28     } catch (JsonProcessingException e) {
29         throw new RuntimeException("Failed to serialize Order
summary", e);
30     }
31 }
32 }
33

```

application.properties

```

1 server.port=8081
2 spring.application.name=kafka-producer-service
3
4 # Both brokers listed for redundancy, if one is down, producer
connects to the other
5 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
6
7 # Metadata refresh interval: how often producer re-fetches cluster
info (default 5 min)
8 spring.kafka.producer.properties.metadata.max.age.ms=300000
9
10 # Asking broker to reply only when all ISR successfully get the events
11 spring.kafka.producer.acks=all
12
13 #-----SERIALIZER-----
14 # Key needs to be String
15 spring.kafka.producer.key-
serializer=org.apache.kafka.common.serialization.StringSerializer
16
17 # Value: custom serializer
18 spring.kafka.producer.value-
serializer=com.eda.producer.serializer.OrderSummarySerializer
19
20 #-----RECORD ACCUMULATOR-----
21 # Batching, wait 20ms to collect more messages before sending
22 spring.kafka.producer.properties.linger.ms=20
23
24 # Max batch size, send when batch reaches 32KB (even if linger.ms
hasn't expired)
25 spring.kafka.producer.properties.batch.size=32768
26
27 # Total buffer memory for all batches: 32MB (if full, send() will get
blocked till space created or max block time)
28 spring.kafka.producer.properties.buffer.memory=33554432
29
30 # Max time send() blocks if buffer is full (default 60s), after this
time, send() call will fail immediately
31 spring.kafka.producer.properties.max.block.ms=60000
32
33
34 #-----COMPRESSION-----
35 # compresses entire batch before sending
36 spring.kafka.producer.properties.compression.type=snappy
37
38
39 #-----SENDER THREAD-----
40 # Retry on transient failures (network timeout, leader change, etc.)
41 spring.kafka.producer.retries=3
42
43 # Max in-flight requests

```

```

44 spring.kafka.producer.properties.max.in.flight.requests.per.connection
    =5
45

```

But what if we have many producer:

- Some need JsonSerializer
- Some need custom Serializer

Then how in 1 application.properties we will define?

```

1 @Bean
2 public KafkaTemplate<String, Order>
   customSerializerKafkaTemplate(KafkaProperties kafkaProperties) {
3
4     // Start with all producer properties from application.properties
5     Map<String, Object> props =
6         new HashMap<>
7         (kafkaProperties.buildProducerProperties(null));
8
9     // Override ONLY the value serializer - everything else stays the
   same
10    props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
   OrderSummarySerializer.class);
11
12    DefaultKafkaProducerFactory<String, Order> factory =
   new
13    DefaultKafkaProducerFactory<>(props);
14
15    return new KafkaTemplate<>(factory);
16 }

```

```

1 import org.springframework.beans.factory.annotation.Autowired;
2 import org.springframework.kafka.core.KafkaTemplate;
3 import org.springframework.kafka.support.SendResult;
4 import org.springframework.stereotype.Service;
5 import java.util.concurrent.CompletableFuture;
6
7 @Service
8 public class OrderProducerService {
9
10    @Autowired
11    private KafkaTemplate<String, Order> kafkaTemplate;
12
13    @Autowired
14    @Qualifier("customSerializerKafkaTemplate")
15    private KafkaTemplate<String, Order>
   customSerializerKafkaTemplate;
16
17    public void sendWithoutKey(Order order) {
18        String key = order.getOrderid();
19
20        CompletableFuture<SendResult<String, Order>> future =
   customSerializerKafkaTemplate.send("order-events", order);
21
22        future.whenComplete((result, ex) -> {
23            if (ex == null) {

```

```

24         System.out.println("Order event sent
    successfully");
25         System.out.println("Partition: " +
    result.getRecordMetadata().partition());
26         System.out.println("Offset: " +
    result.getRecordMetadata().offset());
27     } else {
28         System.out.println("Failed to send order
    event: " + ex.getMessage());
29     }
30 }
31 }
32 }
33

```

In application.properties we can keep the default "**JsonSerializer**"

```

1 server.port=8081
2 spring.application.name=kafka-producer-service
3
4 # Both brokers listed for redundancy, if one is down, producer
    connects to the other
5 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
6
7 # Metadata refresh interval: how often producer re-fetches cluster
    info (default 5 min)
8 spring.kafka.producer.properties.metadata.max.age.ms=300000
9
10 # Asking broker to reply only when all ISR successfully get the events
11 spring.kafka.producer.acks=all
12
13 #-----SERIALIZER-----
14 # Key needs to be String
15 spring.kafka.producer.key-
    serializer=org.apache.kafka.common.serialization.StringSerializer
16
17 # Value: JsonSerializer, auto-converts Java objects to JSON bytes
18 spring.kafka.producer.value-
    serializer=org.springframework.kafka.support.serializer.JsonSerializer
19
20 #-----RECORD ACCUMULATOR-----
21 # Batching, wait 20ms to collect more messages before sending
22 spring.kafka.producer.properties.linger.ms=20
23
24 # Max batch size, send when batch reaches 32KB (even if linger.ms
    hasn't expired)
25 spring.kafka.producer.properties.batch.size=32768
26
27 # Total buffer memory for all batches: 32MB (if full, send() will get
    blocked till space created or max block time)
28 spring.kafka.producer.properties.buffer.memory=33554432
29
30 # Max time send() blocks if buffer is full (default 60s), after this
    time, send() call will fail immediately
31 spring.kafka.producer.properties.max.block.ms=60000
32
33
34 #-----COMPRESSION-----
35 # compresses entire batch before sending
36 spring.kafka.producer.properties.compression.type=snappy
37
38
39 #-----SENDER THREAD-----

```

```
40 # Retry on transient failures (network timeout, leader change, etc.)
41 spring.kafka.producer.retries=3
42
43 # Max in-flight requests
44 spring.kafka.producer.properties.max.in.flight.requests.per.connection
45 =5
```

Now, last part, we discussed that, when:

max.in.flight.requests.per.connection > 1

and

Retries is also > 0

Then there is risk of message reordering.

Ex:

Request A → in-flight

Request B → in-flight

Request A → fails

Request B → success (written successfully in partition)

Request A → retries and Success (written after Request B)

How to resolve this Risk?